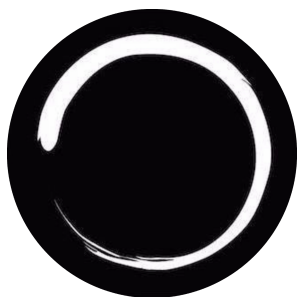


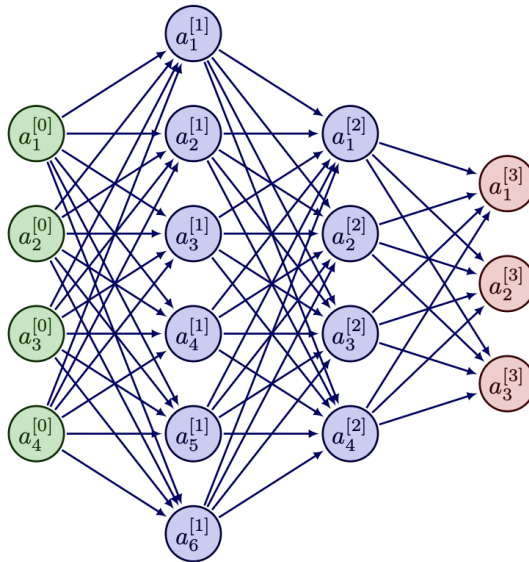
inzva DLSG

Homework 1



Open Questions

1. What is **deep learning**, and how does it differ from **traditional machine learning**?
2. What do we mean by **forward propagation**?
3. What is **backpropagation**, and why is it essential for neural networks?
4. Why do we use **activation functions** for the layers? What happens if we don't use activation functions?
5. Write **forward propagation equations** and corresponding dimensions of the **matrices** and **vectors** for the neural network below. Assume your batch size is 1. (Dimensions for weight matrices and input vectors for each layer, as we did in the session.)



6. When does our model have **underfitting** and **overfitting**? How do we determine these situations? Evaluate your answer.
7. What is the **learning rate** in the context of training neural networks? What happens if the learning rate is too high or too low?
8. What is **dropout**, and how does it help in training neural networks?
9. What is the **main difference** between model's weights and **hyperparameters**?

Coding Assignment

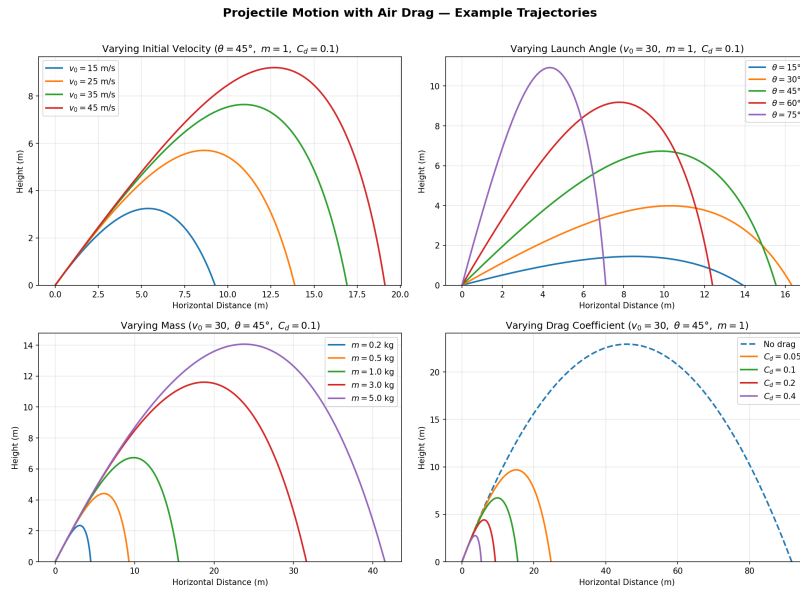


Figure 1: Projectile trajectories showing the effect of varying each input parameter.

You will be working with a partially completed deep learning project that focuses on training a Multi-Layer Perceptron (MLP) to approximate a physical system using PyTorch. Specifically, the model predicts the **horizontal distance** a projectile travels given its launch parameters, where quadratic air drag makes the problem non-trivial (no closed-form solution exists). The provided code includes structured comments and placeholder variables that you are required to fill out based on the instructions within the code. Your task is to define the model architecture, implement the training loop, and complete the experiment runner functions.

The `main.py` file serves as the entry point for the project and walks you through the steps of data loading, model initialization, training, and testing. The comments provided in the code will guide you in filling out necessary parts where values or function implementations are missing. There are additional utility files that handle the loading and processing of data as well as visualization tools to help you monitor model performance during training.

It is important to note that the goal of this assignment is not to achieve the lowest possible loss, but rather to successfully train, test, and visualize your results using the provided codebase. Focus on understanding the workflow, completing the missing parts, and ensuring that the model can run without errors. Pay close attention to the provided comments, and ensure that your code is well-structured and functional.

Make sure to follow the instructions in the code carefully and utilize the PyTorch documentation if needed. Once the necessary parts are implemented, you should be able to train the MLP, visualize its performance, and evaluate it on the test set. You will also run three experiment sets comparing optimizers, regularization methods, and dataset sizes.

The Physics

A projectile is launched with initial velocity v_0 (10–50 m/s), launch angle θ (5° – 80°), mass m (0.1–5.0 kg), and drag coefficient C_d (0.01–0.5). The equations of motion are

governed by gravity and quadratic air drag:

$$\vec{F}_{\text{drag}} = -C_d |\vec{v}| \vec{v}, \quad \vec{F}_{\text{gravity}} = -m g \hat{y}$$

The acceleration in each component is:

$$a_x = -\frac{C_d}{m} |\vec{v}| v_x, \quad a_y = -g - \frac{C_d}{m} |\vec{v}| v_y$$

where $|\vec{v}| = \sqrt{v_x^2 + v_y^2}$. The trajectory is computed via numerical integration (Euler's method with small Δt). Your MLP must learn the mapping:

$$f(v_0, \theta, m, C_d) \longrightarrow d \text{ (horizontal distance)}$$

Project Structure

assignment2/

<code>config.py</code>	— Hyperparameter configuration (modify for experiments)
<code>data.py</code>	— Synthetic data generation (projectile physics)
<code>model.py</code>	— MLP model definition (TODO: complete the model)
<code>train.py</code>	— Training & validation loop (TODO: complete the loop)
<code>evaluate.py</code>	— Evaluation & experiment runner (TODO: complete)
<code>visualize.py</code>	— Plotting utilities (provided, no changes needed)
<code>main.py</code>	— Entry point, run this file

Required Experiments

You must complete **3 experiment sets** and analyze the results:

Experiment	What to Compare
Experiment 1: Optimizers	SGD+Momentum vs. Adam — convergence speed, final loss
Experiment 2: Regularization	No regularization vs. Dropout vs. Weight Decay
Experiment 3: Dataset Size	Small (500) vs. Medium (2000) vs. Large (10000) samples

Files to Complete

1. `model.py` — Define the MLP architectures (`PhysicsMLP` and `PhysicsMLPWithDropout`).
2. `train.py` — Implement the training loop, validation loop, and optimizer selection.
3. `evaluate.py` — Complete the experiment runner functions for each experiment set.

Tips

- Start by reading through **all** files before writing any code.
- Complete `model.py` first, then `train.py`, then `evaluate.py`.

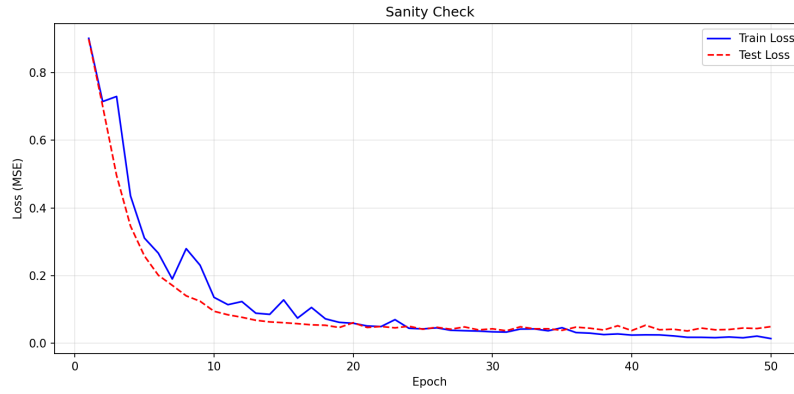


Figure 2: Training and validation loss curves from the sanity check run.

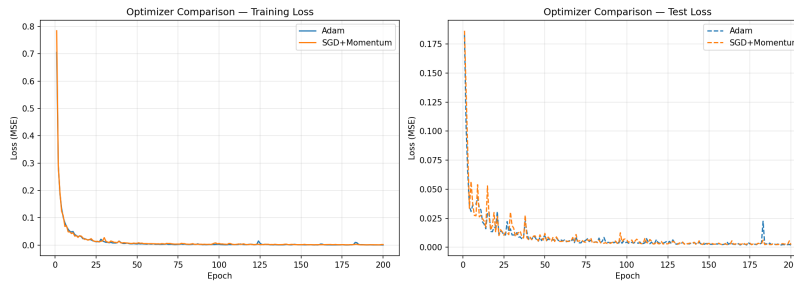


Figure 3: Optimizer comparison — SGD+Momentum vs. Adam.

- Use `config.py` to change hyperparameters — do not hardcode values.
- The `visualize.py` file is fully provided — use its functions to generate plots.
- Set random seeds for reproducibility.

Notebook

Also you have a notebook to complete. You can find the instructions in the notebook.

Bonus Questions (Optional)

1. Assume you have a linear regression model $\hat{y} = \mathbf{f}_{\mathbf{w}}(x) = \mathbf{w}_1 x + \mathbf{w}_0$ for a dataset of pairs $(x_i, y_i)_{1 \leq i \leq N}$. You want to estimate your weights so that your model fits the data well. You have residuals (errors of your model for every pair) $e_i = (y_i - \hat{y}_i)$. Then, using the residual sum of squares (RSS) function $\mathbf{RSS} = \sum_{i=1}^N e_i^2$, show that your estimated weights can be found as

$$\mathbf{w}_1 = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}$$

$$\mathbf{w}_0 = \bar{y} - \mathbf{w}_1 \bar{x}$$

where $\bar{x} = \frac{1}{n} \sum_{i=1}^N x_i$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^N y_i$ are the means of your samples.

2. In binary classification, where the output can either be 0 or 1, the binary cross-entropy loss function is used. The BCE loss measures the difference between the true labels and the predicted probabilities. Given a single data point, the binary cross-entropy loss can be defined as:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where:

- y is the true label (0 or 1).
- $\hat{y}$ is the predicted probability for the positive class (1).

In forward propagation, we calculate the predicted probability using a logistic function (also known as the sigmoid function), followed by the binary cross-entropy loss.

For a model output z (logit):

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This function converts the raw score (logit) z into a probability $\hat{y}$ between 0 and 1. Then assume you have the following neural network with 2 hidden layer and sigmoid function for every layer's activation function. Derive the **forward** and **backward propagation** equations for the binary cross-entropy loss function.

